

# Algoritmos e Estruturas de Dados

**Fernando Neto**  
**fernando@cin.ufpe.br**

Conteúdo modificado dos slides cedidos pelos professores  
Hansenclever Bassani e Renato Vimieiro



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

CIn.ufpe.br

# Programação Dinâmica



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

CIn.ufpe.br

## Objetivo da aula

- Definir o que é a técnica de Programação Dinâmica
- Enumerar suas características e quando aplicá-la
- Dar exemplos de problemas que podem ser resolvidos
- Comparar Programação dinâmica com Problemas Gulosos

## Introdução

- Até o momento, estudamos as estratégias dividir-e-conquistar e gulosa para especificação de algoritmos
- Essas estratégias são úteis para solucionar diversos problemas práticos
- No entanto, elas não são a solução para todos os problemas
- Estudaremos nesta aula uma nova estratégia para desenho de algoritmos chamada **programação dinâmica**

## Introdução

- A técnica de programação dinâmica foi proposta pelo matemático estadunidense Richard Bellman nos anos 1950s para **solucionar problemas de otimização**
- Ele usou a palavra programação para se referir ao **ato de planejar as ações a serem tomadas** e não ao ato de escrita de código de computador
- Apesar de conceitualmente diferente, programação dinâmica **assemelha-se** a dividir-e-conquistar por resolver problemas maiores através de subproblemas

## Por que o nome “Programação Dinâmica”?

- “...Os anos 50 não foram bons anos para pesquisa matemática. O Secretário de Defesa ...tinha um medo patológico e odiava a palavra ‘busca’ ou ‘pesquisa’... Eu portanto decidi utilizar a palavra, **‘Programação’**. Eu queria passar a ideia de que era dinâmico, multi-estágio... eu pensei, vamos ... escolher uma palavra que tem um significado absolutamente preciso, ou seja, **dinâmico**... é impossível usar a palavra, **dinâmico**, em um sentido pejorativo. Tente pensar em alguma combinação que vai dar um sentido pejorativo. É impossível. Assim, eu pensei que programação dinâmica era um bom nome. Era algo que nem mesmo um congressista poderia opor-se.”

Richard Bellman, “Eye of the Hurricane: an autobiography” 1984.

# Introdução

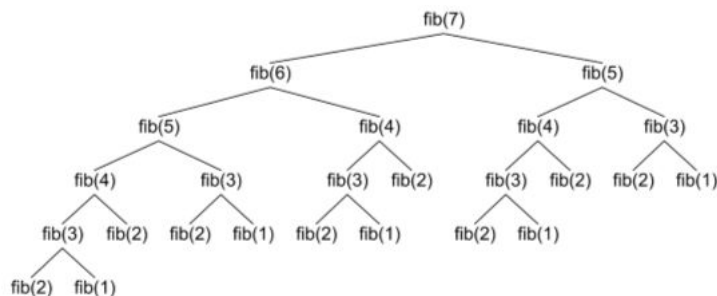
- A ideia geral da estratégia dividir-e-conquistar é particionar o problema original em problemas menores independentes, resolvê-los e combinar os resultados para obter a solução global
- A **programação dinâmica** é útil nos casos em que os subproblemas **não são independentes**
- Exemplo:
  - Sequência de Fibonacci:  $F(n) = F(n-1) + F(n-2)$
  - **Os subproblemas não são independentes.** Logo, muitos são resolvidos diversas vezes, repetindo esforço e gerando um custo desnecessário
- A **programação dinâmica faz uso de tabelas que armazenam resultados intermediários, contornando esse problema**

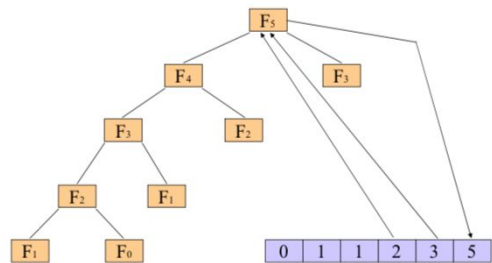
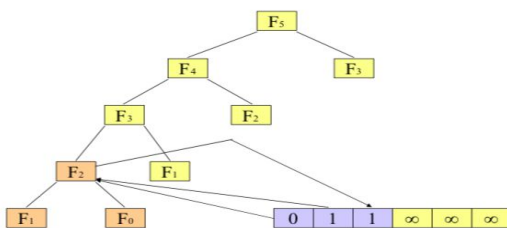
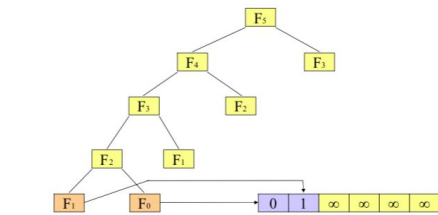
## Exemplo do Fibonacci

- $F(n) = F(n-1) + F(n-2)$

|      |   |   |   |   |
|------|---|---|---|---|
| i    | 0 | 1 | 2 | 3 |
| F(i) | 0 | 1 | 1 | 2 |

$$\begin{aligned} F(7) &= F(6) + F(5) \\ &= (F(5) + F(4)) + (F(4) + F(3)) \end{aligned}$$





## Fibonacci com Programação Dinâmica

```
def fib(x):
    if x==0:
        return 0
    elif x==1:
        return 1
    else:
        return fib(x-1) + fib(x-2)
```

```
def fibDinamico(x, dic={}):
    if x==0:
        return 0
    elif x==1:
        return 1
    else:
        if (x+1 in dic):
            x1 = dic[x+1]
        else:
            x1 = fibDinamico(x-1, dic)
        if (x+2 in dic):
            x2 = dic[x+2]
        else:
            x2 = fibDinamico(x-2, dic)
        x = x1 + x2
        dic[x] = x
        return x
```

```
In [7]: dic={}
```

```
In [8]: %%time
print(fibDinamico(40,dic))
```

```
35372936
CPU times: user 2.89 s, sys: 7.76 ms, total: 2.9 s
Wall time: 2.91 s
```

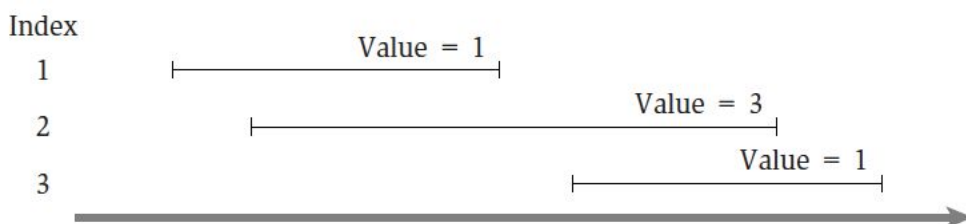
```
In [9]: %%time
print(fib(40))
```

```
102334155
CPU times: user 54.8 s, sys: 188 ms, total: 55 s
Wall time: 55.6 s
```

## Problema do agendamento de compromissos

- Vejamos um exemplo:
- Suponha um problema em que seja necessário escolher, dentre **diversos compromissos**, aqueles que geram **o maior valor agregado**
  - Todo compromisso tem um horário de início e fim; eles não podem se sobrepor
  - Todo compromisso está associado a um valor (lucro, prioridade, ...)
  - O objetivo é escolher os compromissos de forma a maximizar a soma desses valores

## Problema do agendamento de compromissos

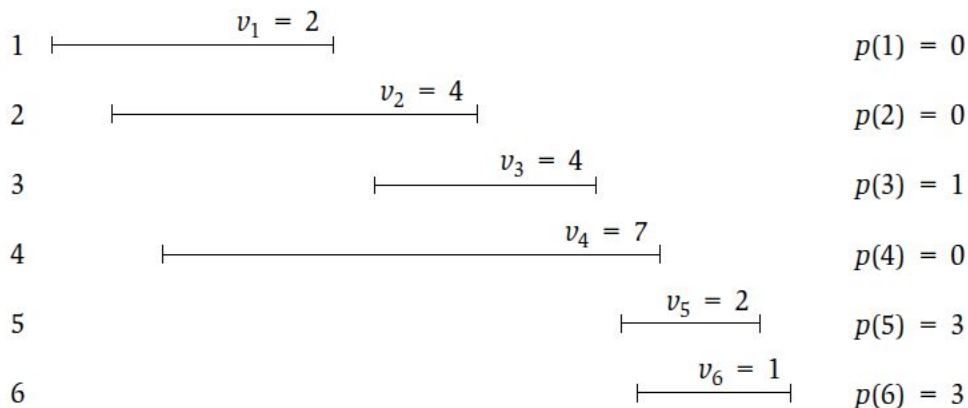


## Problema do agendamento de compromissos

- Se rotularmos os compromissos de 1 a  $n$  ordenados pelo horário de término, o problema se torna:
  - Encontrar  $S \subseteq \{1, 2, \dots, n\}$  tal que  $\sum_{i \in S} v(i)$  é máximo
  - Onde  $v(i)$  é o valor do compromisso  $i$
  - Para todo  $i$ ,  $\text{fim}(i) < \text{início}(i+1)$
- Para solucionar o problema, definimos uma função  $p(j)$  “*antecessor imediato de  $j$* ” como sendo o maior valor de  $i$  tal que  $i < j$  ( $\text{fim}(i) < \text{início}(j)$ )
  - $p(j) = 0$  se não existir um compromisso com fim anterior ao início de  $j$

## Problema do agendamento de compromissos

Index



## Problema do agendamento de compromissos

- Seja  $S^*$  uma solução ótima para o problema
- O último compromisso pode ou não pertencer a  $S^*$
- 1. Se  $n$  pertencer a  $S^*$ , então nenhum compromisso entre  $p(n)$  e  $n$  pode pertencer a  $S^*$ 
  - Além disso, se  $n$  pertencer a  $S^*$ , então  $S^*$  deve conter uma solução ótima para o subproblema  $\{1, 2, \dots, p(n)\}$
- 2. Similarmente, se  $n \notin S^*$ , então  $S^*$  deve conter uma solução ótima de  $\{1, 2, \dots, n-1\}$

## Problema do agendamento de compromisso

- As observações anteriores nos levam a definir o valor  $S(j)$  da solução ótima para  $\{1, 2, \dots, j\}$  como
  - $S(j) = \max(v(j) + S(p(j)), S(j-1))$
  - $S(0) = 0$
- Podemos derivar uma função recursiva diretamente dessa definição para computar o valor máximo
- Também podemos afirmar se  $j$  pertence ou não à solução se
  - $v(j) + S(p(j)) \geq S(j-1)$



## Problema do agendamento de compromissos

- O problema de tal algoritmo recursivo é que, assim como a versão recursiva de Fibonacci, vários  $S(i)$  são computados repetidas vezes
  - A complexidade do algoritmo se torna exponencial
- Podemos modificar o algoritmo para ser um pouco mais ‘esperto’ *memorizando* resultados já computados
  - Criamos uma tabela para armazenar  $S(i)$
  - A primeira vez que  $S(i)$  for necessário, computamos seu valor
  - Na próxima vez, retornamos o valor presente na tabela

## Problema do agendamento de compromissos

- A tabela contém um valor para cada  $1 \leq i \leq n$ , além de zero
  - Logo, preenchê-la tem um custo  $O(n)$
- Computados os valores de  $S(i)$ , podemos computar a solução efetiva, mostrando os compromissos a serem cumpridos:
- def compromisso(j):
  - Se  $j = 0$ : retornar
  - Senão:
    - Se  $v(j) + S(p(j)) \geq S(j-1)$ :
      - compromisso( $p(j)$ );
      - imprimir “j”
    - Senão:
      - compromisso( $j-1$ )
- A função compromisso também tem custo  $O(n)$

## Programação dinâmica

- O problema anterior destaca a aplicabilidade de programação dinâmica
- Através do simples armazenamento de soluções parciais, foi possível reduzir drasticamente a complexidade de tempo do algoritmo
- Ainda podemos refinar o algoritmo anterior
  - Podemos transformá-lo em um algoritmo iterativo computando os valores a partir de zero (bottom-up)
- A estratégia de programação dinâmica é exatamente essa ideia

## Programação dinâmica

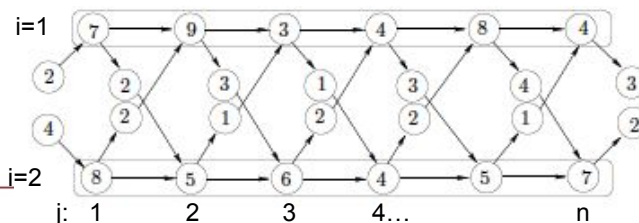
- O desenvolvimento de um algoritmo de programação dinâmica envolve em geral quatro etapas
  1. Caracterizar a estrutura de uma solução ótima
  2. Definir uma relação de recorrência para o valor de uma solução ótima
  3. Calcular o valor de uma solução ótima por um procedimento bottom-up
  4. Construir uma solução ótima a partir dos valores computados

## Programação dinâmica

- A aplicabilidade de programação dinâmica em geral depende da identificação das seguintes características
  - Existe apenas um número polinomial de subproblemas
  - A solução do problema original deve ser facilmente computada através das soluções dos subproblemas
  - Existe uma ordem natural sobre os subproblemas, permitindo resolvê-los do 'menor' para o 'maior'

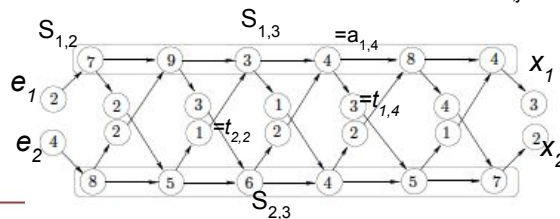
## Programação de uma linha de montagem

- Vamos analisar outro problema e verificar soluções com programação dinâmica para o mesmo:
- Linha de Montagem:
  - Suponha que uma indústria possui duas linhas de montagem com  $n$  estações para fabricar seu produto
  - Cada estação  $1 \leq j \leq n$  de uma linha  $i = 1$  ou  $2$  é denotada por  $S_{i,j}$
  - Cada estação possui um tempo de montagem distinto ( $a_{i,j}$ )



## Programação de uma linha de montagem

- Para montar um produto, suas peças devem ser transportadas do depósito até à linha de montagem
  - O tempo de transporte de entrada é dado por  $e_i$
- Da mesma forma, ao ser finalizado, o produto deve ser transportado para um armazém
  - O tempo de transporte de cada linha até o armazém é  $x_i$
- O produto avança de uma estação para outra com tempo desprezível
  - No entanto, se a próxima estação estiver sobrecarregada, o produto pode ser deslocado para a estação equivalente na outra linha em tempo  $t_{ij}$



## Programação de uma linha de montagem

- O problema consiste em minimizar o tempo necessário para a montagem de um produto
- Em outras palavras, quais estações nas linhas 1 e 2 devem ser escolhidas
  - Um algoritmo exaustivo (força-bruta) teria custo  $O(2^n)$  (cada estação pode ou não fazer parte da solução)
- Podemos fazer melhor usando programação dinâmica. Para isso, devemos seguir as 4 etapas mencionadas anteriormente

## Programação de uma linha de montagem

### 1. Caracterizar a estrutura de uma solução ótima

- A montagem mais rápida para o produto passando pela estação  $S_{1j}$  pode ser facilmente computada para  $j=1$
- Para  $2 \leq j \leq n$ , existem duas opções:
  - O produto veio de  $S_{1,j-1}$  com tempo desprezível de transição; ou
  - O produto veio de  $S_{2,j-1}$  com tempo de transição  $t_{2,j-1}$
- O caminho mais rápido passando por  $S_{1j}$  envolve o caminho mais rápido até  $S_{1,j-1}$  e  $S_{2,j-1}$ 
  - O mesmo raciocínio se aplica a  $S_{2,j}$
- O problema apresenta subestrutura ótima

## Programação de uma linha de montagem

### 2. Solução recursiva

- A solução ótima passando por uma estação em uma linha envolve as soluções ótimas passando pela estação anterior em ambas as linhas
- Seja  $f[i,j]$  o menor tempo gasto na montagem desde a origem até a estação  $S_{ij}$
- A solução ótima do problema  $f^*$  é
  - $f^* = \min(f[1,n] + x_1, f[2,n] + x_2)$
- O tempo necessário até a estação 1 em cada linha é
  - $f[i,1] = e_i + a_{i1}$

## Programação de uma linha de montagem

3. Calcular o valor de uma solução ótima por um procedimento bottom-up:

- O tempo necessário até uma estação  $S_{1j}$

$$f[1,j] = \begin{cases} e_1 + a_{11}, & \text{se } j = 1 \\ \min(f[1,j-1] + a_{1j}, f[2,j-1] + a_{1j} + t_{2j-1}), & \text{se } j \geq 2 \end{cases}$$

- O tempo necessário até uma estação  $S_{2j}$

$$f[2,j] = \begin{cases} e_2 + a_{21}, & \text{se } j = 1 \\ \min(f[2,j-1] + a_{2j}, f[1,j-1] + a_{2j} + t_{1j-1}), & \text{se } j \geq 2 \end{cases}$$

## Programação de uma linha de montagem

4. Construir uma solução ótima a partir dos valores computados

- Podemos novamente derivar um algoritmo recursivo diretamente da definição de menor tempo necessário até uma estação para calcular a solução ótima
  - Mais uma vez, vários valores  $f[i,j]$  serão computados **desnecessariamente** diversas vezes
- Alternativamente podemos aplicar programação dinâmica para solucionar o problema
  - Armazenamos  $f[i,j]$  em uma tabela
  - Computamos os valores para  $j=1$  até  $n$  para as duas linhas iterativamente

## Programação de uma linha de montagem

Montagem(a,t,e,x,n)

$f[1,1] = e[1] + a[1,1]$

$f[2,1] = e[2] + a[2,1]$

para  $j=2$  até  $n$

$f[1,j] = \min(f[1,j-1]+a[1,j], f[2,j-1]+a[1,j]+t[2,j-1])$

$f[2,j] = \min(f[2,j-1]+a[2,j], f[1,j-1]+a[2,j]+t[1,j-1])$

$f^* = \min(f[1,n]+x[1], f[2,n]+x[2])$

## Programação de uma linha de montagem

imprimeCaminho(i,j)

se  $j > 1$

se  $f[i][j] == f[i][j-1] + a[i][j-1]$

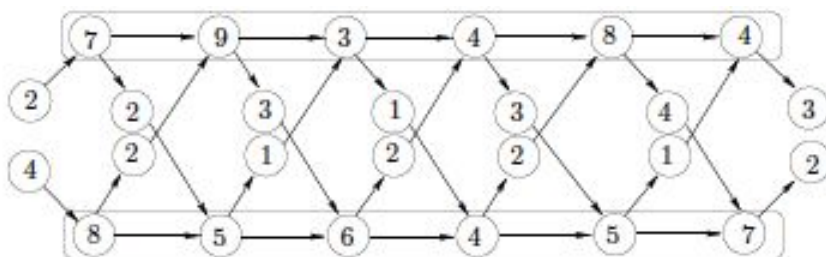
imprimeCaminho(i,j-1)

senão

imprimeCaminho((i+1)%2,j-1)

imprimir “linha” i “estacao” j

## Programação de uma linha de montagem



## Outros problemas resolvidos por PD

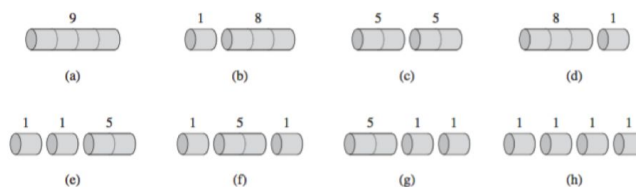
### ■ Corte de Hastes

- Hastes de [aço-5711](http://www.ime.usp.br/~am/5711) são vendidas em pedaços de tamanho inteiro. As usinas produzem hastes longas, e os comerciantes cortam em pedaços para vender.
- Por razões inexplicáveis, o preço de uma haste de tamanho  $i$  está tabelado como  $p_i$ , e para o problema abaixo ter graça, é bom que isso não seja função linear de  $i$ .
- **Problema:** Dada uma haste de tamanho  $n$  e a tabela de preços  $p_i$ , qual a melhor forma de cortar para maximizar o preço de venda total?

| length $i$  | 1 | 2 | 3 | 4 | 5  | 6  | 7  | 8  | 9  | 10 |
|-------------|---|---|---|---|----|----|----|----|----|----|
| price $p_i$ | 1 | 5 | 8 | 9 | 10 | 17 | 17 | 20 | 24 | 30 |

- retirado de:  
<https://www.ime.usp.br/~am/5711/aulas/aula09.pdf>





**Figure 15.2** The 8 possible ways of cutting up a rod of length 4. Above each piece is the value of that piece, according to the sample price chart of Figure 15.1. The optimal strategy is part (c)—cutting the rod into two pieces of length 2—which has total value 10.

## Outros problemas resolvidos por PD

### ■ Multiplicação de Cadeia de Matrizes

- Dada uma sequência de matrizes, o objetivo é encontrar a forma mais eficiente de **multiplicar essas matrizes**. O problema não é na realidade para *realizar* a multiplicação, mas apenas para decidir a sequência de multiplicações das matrizes envolvidas.
- [https://pt.wikipedia.org/wiki/Multiplicação\\_de\\_cadeia\\_de\\_matrizes](https://pt.wikipedia.org/wiki/Multiplicação_de_cadeia_de_matrizes)
- Por exemplo:
- $M_1$  é uma matriz  $10 \times 100$
- $M_2$  é uma matriz  $100 \times 5$
- $M_3$  é uma matriz  $5 \times 50$
- Multiplicar  $M_1 M_2 M_3$  pode ser feito
  - $(M_1 M_2) M_3$  faremos 7500 multiplicações escalares
  - $M_1 (M_2 M_3)$  faremos 75000 multiplicações escalares

## Outros problemas resolvidos por PD

### ■ Sub-sequência mais longa:

- $\langle z_1, \dots, z_k \rangle$  é **sub-sequência** de  $\langle x_1, \dots, x_m \rangle$  se existem índices  $i_1 < \dots < i_k$  tais que
- $z_1 = x_{i_1} \dots z_k = x_{i_k}$

- $\langle 5, 9, 2, 7 \rangle$  é subseqüência de  $\langle 9, 5, 6, 9, 6, 2, 7, 3 \rangle$

- **Z** é **sub-sequência comum** de X e Y

- se **Z** é sub-sequência de X e de Y

Exemplo:

X = ABCBDAB

Y = BDCABA

sub-sequência comum de X e Y = **B C A**

X = ABCBDAB

Y = BDCABA

sub-sequência comum de X e Y = **B D A B**

fonte: [https://www.ime.usp.br/~cris/aulas/11\\_1\\_338/slides/aula13.pdf](https://www.ime.usp.br/~cris/aulas/11_1_338/slides/aula13.pdf)



## Outros problemas resolvidos por PD

### ■ Subsequência comum mais longa

Problema: Encontrar uma **subsequência comum máxima** de X e Y .

Exemplos:

X = ABCBDAB

Y = BDCABA

subsequência comum = B C A

subsequência comum **máxima** = B C A B

Outra subsequência comum **máxima** = B D A B



## Outros problemas resolvidos por PD

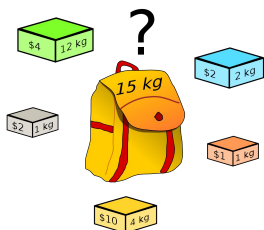
- **Árvore de Busca Binária Ótimas**
  - Criar uma árvore de busca binária cujos elementos possuam prioridade. Os elementos com maior prioridade estarão mais próximos da raiz.
  - **Aplicação:** tradução de textos entre línguas.

## Estratégia Gulosa e Programação dinâmica

- Ambas as estratégias exploram subestruturas ótimas:
  - uma solução ótima para um problema contém soluções ótimas para subproblemas.
  - *Exemplo: o melhor caminho do vértice A a um vértice B no grafo contém os vértices adjacentes {C,D,E,F,G}. Então o melhor caminho entre o vértice C e G é {D,E,F}.*
- Quando usar uma estratégia ou outra?

## Estratégia Gulosa e Programação dinâmica

- Vamos analisar o seguinte problema:
  - Problema da Mochila 0,1
    - O ladrão que assalta uma loja encontra  $n$  itens.
    - O  $i$ -ésimo item vale  $v[i]$  reais e pesa  $w[i]$ .
      - $v[i]$  e  $w[i]$  são inteiros
    - Ele deseja levar consigo uma carga mais valiosa possível, mas só pode carregar no máximo  $W$  quilos em sua mochila, sendo  $W$  um número inteiro.
    - Que itens ele vai levar?
    - O problema se chama 0,1 porque, para cada item, o ladrão tem que decidir se o carrega consigo ou não. Ele não pode levar uma parte fracionária do item.

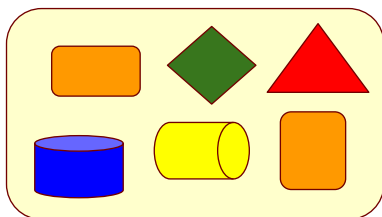


## Estratégia Gulosa e Programação dinâmica

- No **problema fracionária da mochila**, a configuração é a mesma, mas o ladrão pode levar frações de itens, em vez de ter de fazer uma escolha binária (0-1) para cada item.
- Diferença: os itens do problema da mochila 0-1 seriam anéis de ouro, colares, etc, enquanto itens no problema fracionária da mochila seria ouro em pó, etc.
- Ambos os problemas exibem a propriedade de subestrutura ótima.

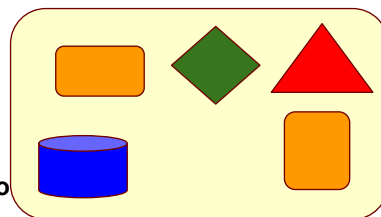
## Estratégia Gulosa e Programação dinâmica

- No problema 0-1, considere que a carga (conjunto de itens) mais valiosa pesa no máximo  $W$  quilos (que é o que ele suporta).
- Se removermos o item  $j$  dessa carga, a carga restante deverá ser a carga mais valiosa que pese no máximo  $W - w[j]$ , que o ladrão pode levar dos  $n-1$  itens originais, excluindo  $j$ .



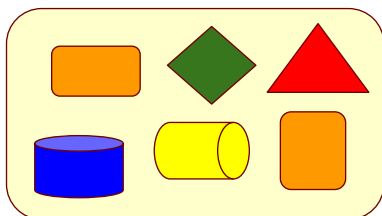
> Carga mais valiosa  
pesando no máximo  $W$

Carga mais valiosa  
pesando no máximo  
 $W - w[j]$



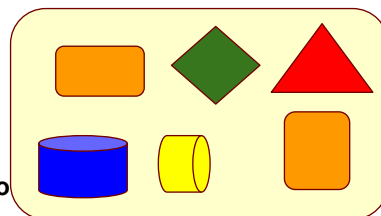
## Estratégia Gulosa e Programação dinâmica

- Por comparação, no caso do problema fracionário, considere que, se removermos um peso  $w$  de um item  $j$  da carga ótima, a carga restante deverá ser a carga mais valiosa que pese no máximo  $W - w$  que o ladrão pode levar dos  $n-1$  itens originais, mais  $w[j] - w$  quilos do item  $j$ .



> Carga mais valiosa  
pesando no máximo  $W$

Carga mais valiosa  
pesando no máximo  
 $W - w$



## Estratégia Gulosa e Programação dinâmica

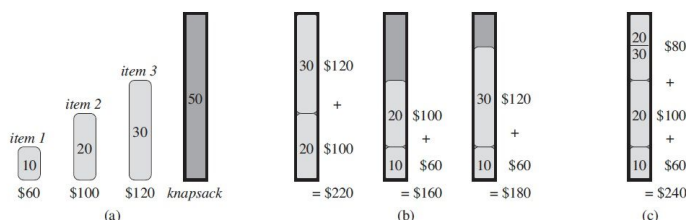
- Ambas as estratégias exploram subestruturas ótimas:
  - uma solução ótima para um problema contém soluções ótimas para subproblemas.
  - *Exemplo: o melhor caminho do vértice A a um vértice B no grafo contém os vértices adjacentes {C,D,E,F,G}. Então o melhor caminho entre o vértice C e G é {D,E,F}.*
- Quando usar uma estratégia ou outra?

## Estratégia Gulosa e Programação dinâmica

- O problema fracionário da mochila pode ser resolvido por uma estratégia gulosa, mas o problema 0-1 não.
- Vejamos como:
- Primeiro, ordenamos os itens por seu valor por kg:  $v[i]/w[i]$ .
- O ladrão começa a pegar o máximo possível do item mais caro. Se ele se esgotar, ele pega o item seguinte mais caro e assim sucessivamente.
- Custo: custo para ordenar  $O(n \log n)$ .

## Estratégia Gulosa e Programação dinâmica

- O problema 0-1 não funciona com estratégia gulosa.
- Veja esse exemplo (3 itens e uma mochila de 50kg)



Valor do item 1: 6 reais/kg, Valor do item 2: 5 reais/kg, valor do item 3: 4 reais/kg.

A estratégia gulosa levaria o item 1 o primeiro.

Porém, a solução ótima não contém o item 1 (ver figura b).

No problema fracionário, se os itens pudessem ser levados parcialmente, realmente a estratégia gulosa funciona escolhendo o item com valor/kg mais caro (ver figura c).

## Estratégia Gulosa e Programação dinâmica

- No problema 0-1, **não** funciona escolher o **melhor** item no momento porque o ladrão não consegue encher sua mochila até a capacidade máxima, e o espaço vazio reduz o valor efetivo por quilo de sua carga.
- No problema 0-1, quando consideramos incluir um item na mochila, temos de comparar a solução para o subproblema que incluía tal item com a solução para o subproblema que excluía esse mesmo item, antes de podermos fazer a escolha.

## Estratégia Gulosa e Programação dinâmica

- Propriedade da escolha gulosa: podemos montar uma solução globalmente ótima fazendo escolhas gulosas locais ótimas.
- Em outras palavras, quando estamos considerando qual escolha fazer, escolhemos a que parece melhor para o problema em questão.
- Na programação dinâmica, fazemos uma escolha em cada etapa, mas, normalmente a escolha depende das soluções para subproblemas. Consequentemente, em geral, resolvemos problemas de programação dinâmica de baixo para cima, passando de subproblemas menores para subproblemas maiores.
- Em um algoritmo guloso, fazemos qualquer escolha que pareça melhor no momento e depois resolvemos o subproblema que resta. A escolha feita por um algoritmo guloso pode depender das escolhas até o momento em questão, mas não pode depender de nenhuma escolha futura ou das soluções para subproblemas.

## Estratégia Gulosa e Programação dinâmica

- Assim, diferentemente da programação dinâmica, que resolve os subproblemas antes de fazer a primeira escolha, um algoritmo guloso faz sua primeira escolha antes de resolver qualquer subproblema. Um algoritmo de programação dinâmica age de baixo para cima, ao passo que, uma estratégia gulosa age de cima para baixo, fazendo uma escolha gulosa após outra, reduzindo cada instância do problema dado a uma instância menor.
- É preciso provar nos problemas resolvidos por um algoritmo guloso que uma escolha gulosa produz uma solução globalmente ótima.



## Leitura

- Capítulo 15 (CLRS)
- Capítulo 6 (Kleinberg e Tardos)
- Capítulo 8 (Livitin)



## Algoritmos e Estruturas de Dados



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

[Cln.ufpe.br](http://Cln.ufpe.br)